Name : AKASH SAMPATH

Topic : Project

Question :

Run Frontend and Backend (Python) + MongoDB using Docker Compose : All are controlled from one docker-compose.yml

Tool : Using Visual Studio Code

Architecture :

docker-compose

├── frontend (React)

├── backend (Python FastAPI)

└── mongodb (MongoDB)

Note : Everything runs locally , no installs except Docker

Procedure :

Step 1 : Install Required Software :

Open virtual studio and do the following commands on terminal :

- `docker –version`
- `docker compose version`
- `node -v`

Output :

```
PS C:\Users\akash\OneDrive\Desktop\Project3> docker --version
Docker version 29.2.0, build 0b9d198
PS C:\Users\akash\OneDrive\Desktop\Project3> docker compose version
Docker Compose version v5.0.2
PS C:\Users\akash\OneDrive\Desktop\Project3> node -v
v24.14.1
PS C:\Users\akash\OneDrive\Desktop\Project3>
```

Step 2 : Create project folder and do the following commands : In terminal :

- `mkdir docker-full-app`
- `cd docker-full-app`
- `mkdir frontend`
- `mkdir backend`

Output :

```
PS C:\Users\akash\OneDrive\Desktop\Project3> mkdir docker-full-app


    Directory: C:\Users\akash\OneDrive\Desktop\Project3


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        29-03-2026     13:52                docker-full-app


PS C:\Users\akash\OneDrive\Desktop\Project3> cd docker-full-app
PS C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app> mkdir frontend


    Directory: C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        29-03-2026     13:52                frontend


PS C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app> mkdir backend
```

```
PS C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app> mkdir backend

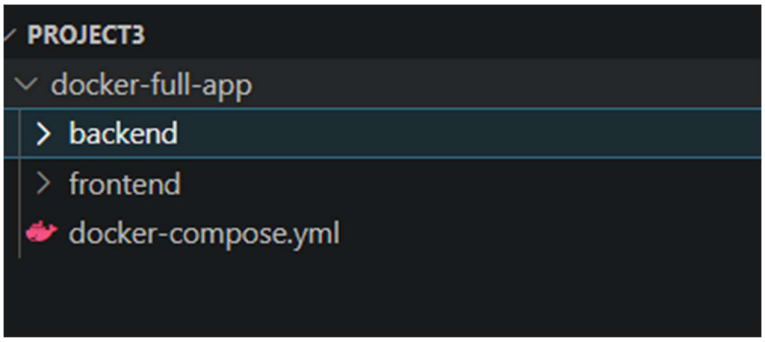

    Directory: C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        29-03-2026     13:52                backend


PS C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app>
```

Note : In `docker-full-app` : Create a file as `docker-compose.yml`

Output : Structure :



Step 3 : Create backend and do the following commands : Based on python :

Command :

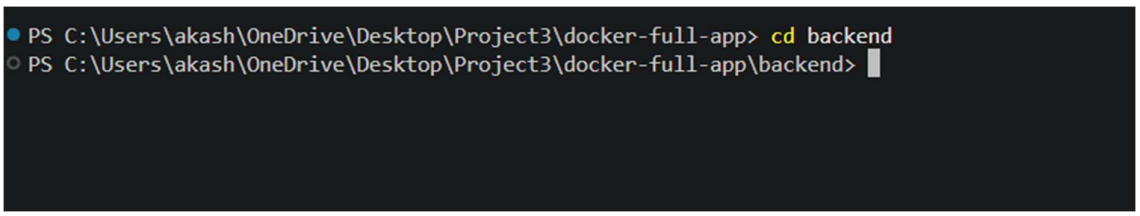
- `cd backend`

Procedure :

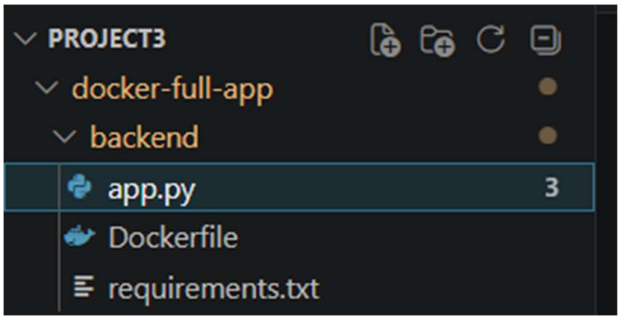
- In backend folder create a file name app.py and paste the code and click save all
- Inside backend folder create another file as requirements.txt and paste the code and click save all
- Inside backend folder create a file as Dockerfile and paste the code and click save all

Output :

- Go inside backend :



- Creation of file of requirements.txt and Dockerfile :



Step 4 : Create frontend and do the following commands :

Procedure :

Open new terminal and do the following commands:

- `cd docker-full-app`
- `cd frontend`
- `npx create-react-app .` (Click yes)

Output :

```
PS C:\Users\akash\OneDrive\Desktop\Project3> cd docker-full-app
PS C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app> cd frontend
PS C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app\frontend> npx create-react-app .
Need to install the following packages:
create-react-app@5.1.0
Ok to proceed? (y)

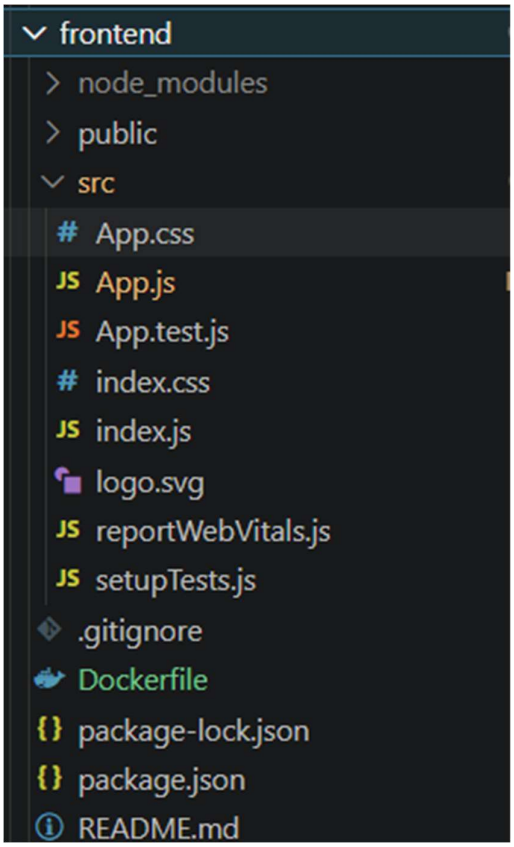
Creating a new React app in C:\Users\akash\OneDrive\Desktop\Project3\docker-full-app\frontend.
```

In frontend :

- Go inside src – App.js and paste code and save all
- In frontend create a file as Dockerfile and paste the code and save all

Output :

- Creation of file as App.js and creation of file as Dockerfile : In frontend



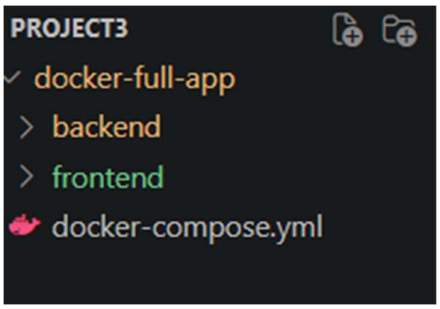
Step 5 : Create docker-compose.yml :

Procedure :

- Create a separate file as docker-compose.yml outside frontend and backend folder and create inside the project folder
- Inside docker-compose.yml paste the code and save all

Output :

- Creation of docker-compose.yml file : Outside : Frontend and Backend : Inside the project folder



Step 6 : Start the entire application from project folder : By doing the following commands :

Procedure :

Open terminal and do the following commands :

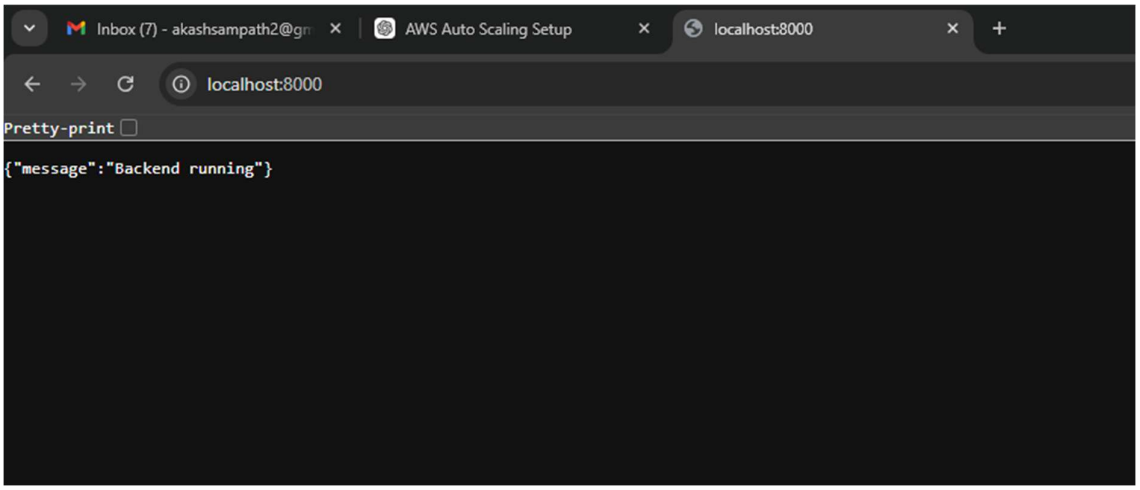
- `cd docker-full-app`
- `cd docker compose up --build`

After doing the following commands above :

This will start frontend and backend with localhost :

- http://localhost:3000 (Frontend)
- http://localhost:8000 (Backend)

Output : Backend :



Output : Frontend :

- Running but after running runtime error coming

Step 7 : Run in background and check running containers and view logs :

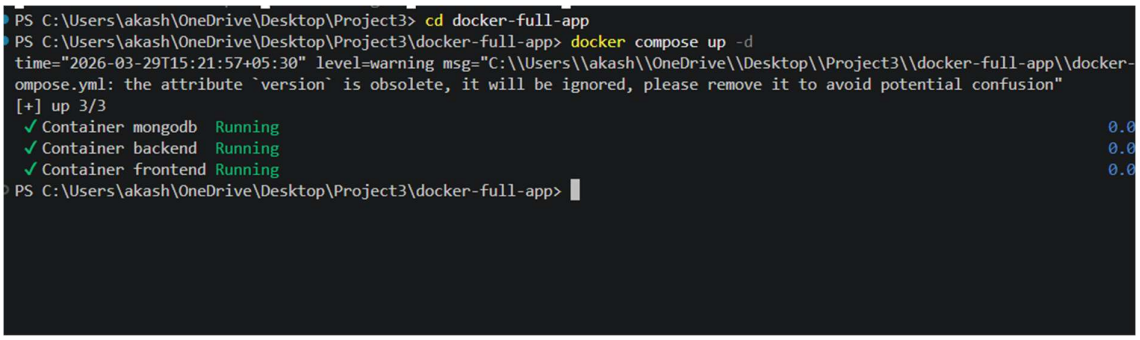
Procedure :

Open terminal and do the following commands :

For running in background :

- `docker compose up -d`

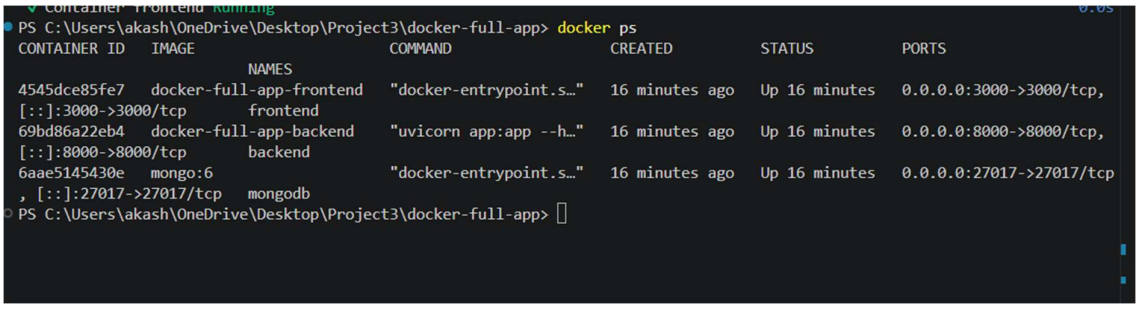
Output : Running in background :



For checking running containers :

- `docker ps`

Output : Checking running containers :

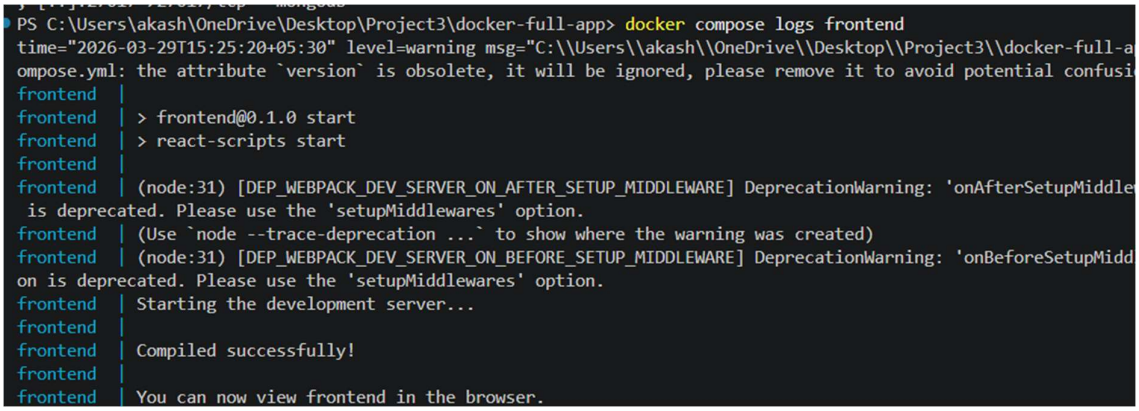


For viewing logs :

For frontend :

- `docker compose logs frontend`

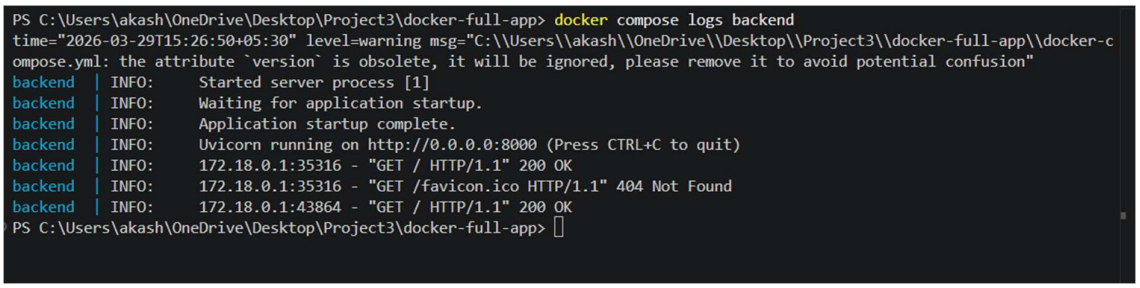
Output : Checking logs in frontend :



For backend :

- `docker compose logs backend`

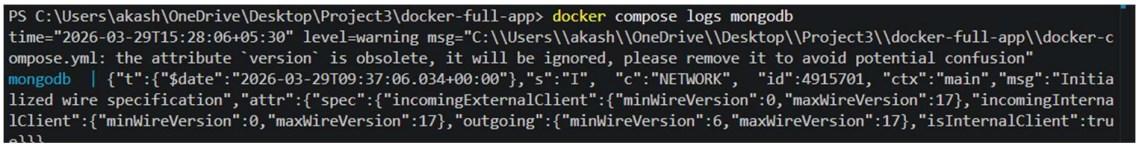
Output : Checking logs in backend :



For MongoDB :

- `docker compose logs mongodb`

Output : Checking logs in MongoDB :



Important Step : Step 8 :

Using MongoDB inside Docker Compose with your Frontend + Backend + MongoDB setup :

Procedure : In backend :

- Check the same code in your docker-compose.yml based on MongoDB
- Now connect backend using app.py to MongoDB replacing the current code with new code and Paste the new code
- Click save all and do the following commands with the new code

Old code : backend/app.py :

```
from fastapi import FastAPI
from pymongo import MongoClient
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

client = MongoClient("mongodb://mongodb:27017/")
db = client["testdb"]
collection = db["items"]

@app.get("/")
def read_root():
    return {"message": "Backend running"}

@app.get("/data")
def get_data():
```

```
    return [{"name": "Docker"}, {"name": "MongoDB"},
{"name": "React"}]
```

New Code : backend/app.py : To connect backend to MongoDB :

```
from fastapi import FastAPI
from pymongo import MongoClient

app = FastAPI()

# connect to mongodb container
client = MongoClient("mongodb://mongodb:27017")

db = client["mydatabase"]
collection = db["products"]



@app.get("/")
def home():
    return {"message": "Backend running"}



# insert data
@app.get("/insert")
def insert():
    data = {"name": "Docker", "type": "DevOps"}
```

```
    collection.insert_one(data)
    return {"message": "Inserted"}


# get data
@app.get("/data")
def get_data():
    data = list(collection.find({}, {"_id": 0}))
    return data
```

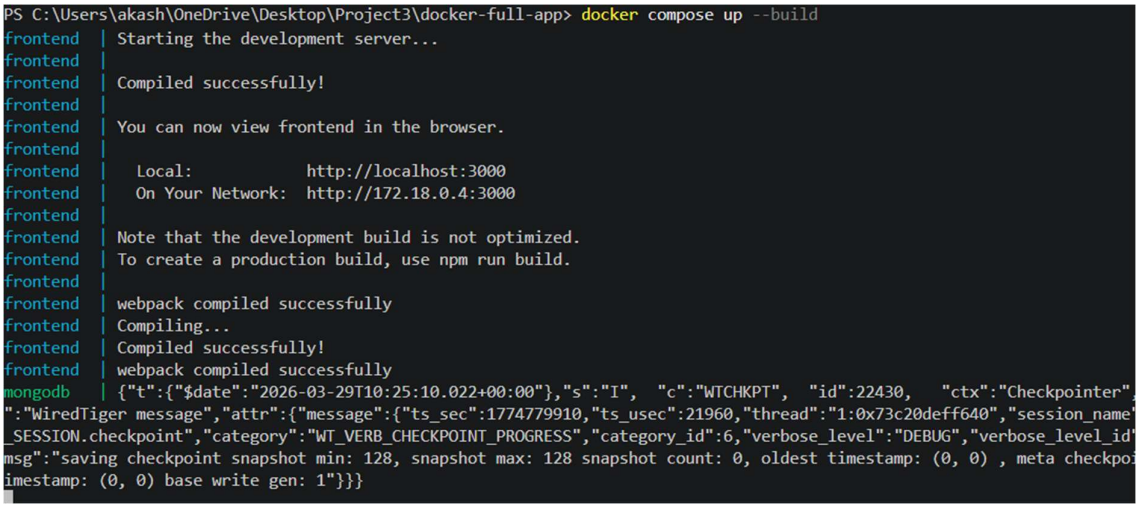
Procedure Continued : General :

- Open new terminal and do the following commands

Commands :

```
cd docker-full-app
docker compose up --build
```

Output :



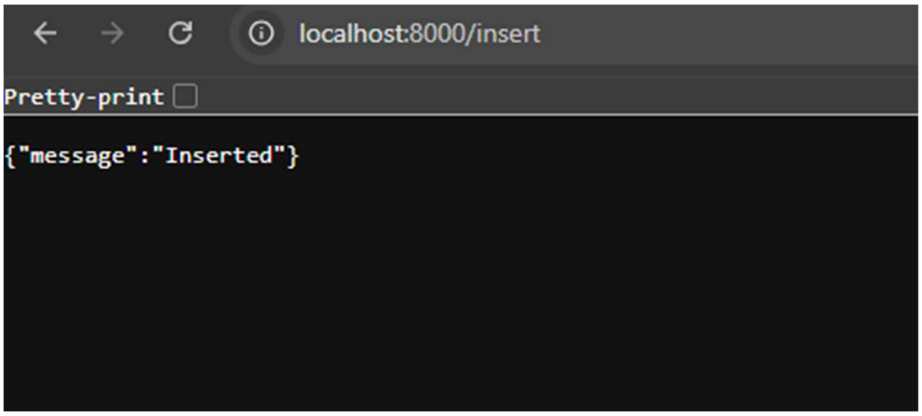
Procedure Continued : (Backend)

After doing the above commands we have to insert data into the MongoDB by opening the browser with the URL :

- http://localhost:8000/insert

Output : (Backend)

We have inserted the message using this URL and new code in backend in app.py file

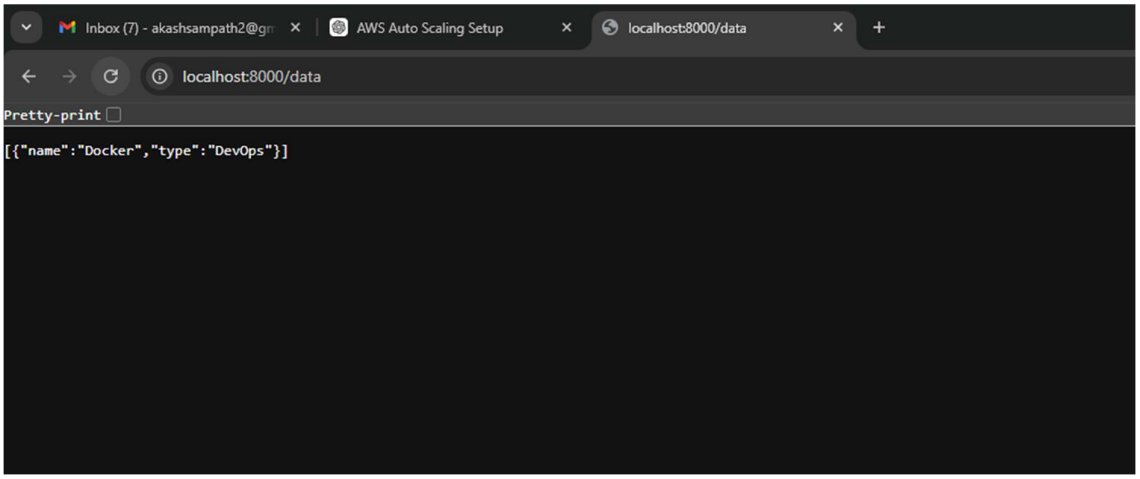


Procedure Continued : (Backend)

Now we have to read the MongoDB data by opening browser with the URL :

- http://localhost:8000/data

Output : We have read the MongoDB data : (Backend)



In Step 9 : We have to connect frontend to MongoDB via Backend :

Procedure :

Open frontend folder – src – App.js and use the new code instead of new code

Old code : App.js : Frontend :

```
import React, { useEffect, useState } from "react";

function App() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("http://backend:8000/data")
      .then(res => res.json())
      .then(data => setData(data));
  }, []);

  return (
    <div>
      <h1>Frontend Running</h1>
      {data.map((item, i) => (
        <p key={i}>{item.name}</p>
      ))}
    </div>
  );
}

export default App;
```

New code : Frontend : App.js :

```
import React, { useEffect, useState } from "react";

function App() {

  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("http://backend:8000/data")
      .then(res => res.json())
      .then(data => setData(data));
  }, []);

  return (
    <div>
      <h1>MongoDB Data</h1>

      {data.map((item, index) => (
        <div key={index}>
          <h3>{item.name}</h3>
          <p>{item.type}</p>
        </div>
      ))}

    </div>
  );
}
```

```
export default App;
```

Procedure Continued : Frontend :

- After pasting the new code in frontend – App.js
- Save all and do the following commands

Open new terminal and do the following commands : General :

```
cd docker-full-app
```

```
docker compose down
```

```
docker compose up –-build
```

Procedure Continued : Frontend :

- Open the frontend using this URL : http://localhost:3000
- Its opening and showing runtime error

Step 10 : Check MongoDB manually :

Procedure :

Open new terminal and do the following commands :

- `docker exec -it mongodb mongosh`
- `show dbs`
- `use mydatabase`
- `show collections`
- `db.products.find()`

Output : MongoDB : Starting : Manually :

```
PS C:\Users\akash\OneDrive\Desktop\Project3> docker exec -it mongodb mongosh
Current Mongosh Log ID: 69c905a7466a2ffb0f8de665
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.1
0
Using MongoDB:          6.0.27
Using Mongosh:          2.5.10

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/le
gal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-29T10:50:44.477+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See h
ttp://dochub.mongodb.org/core/prodnotes-filesystem
   2026-03-29T10:50:45.943+00:00: Access control is not enabled for the database. Read and write access to data and configur
ation is unrestricted
   2026-03-29T10:50:45.945+00:00: vm.max_map_count is too low
------

test>
```

Output :

MongoDB Working Fully : Based on new backend code : App.js : Connection

```
test> show dbs
admin       40.00 KiB
config      92.00 KiB
local       72.00 KiB
mydatabase  40.00 KiB
test> use mydatabase
switched to db mydatabase
mydatabase> show collections
products
mydatabase> db.products.find()
[
  {
    _id: ObjectId('69c8fedd1b100f3f1fd953a0'),
    name: 'Docker',
    type: 'DevOps'
  }
]
```